

Monitoring d'une infrastructure Kubernetes avec Datadog

Évaluation du module Datadog — Formation DevOps (DataScientest)

Imad Sabri

Octobre 2026

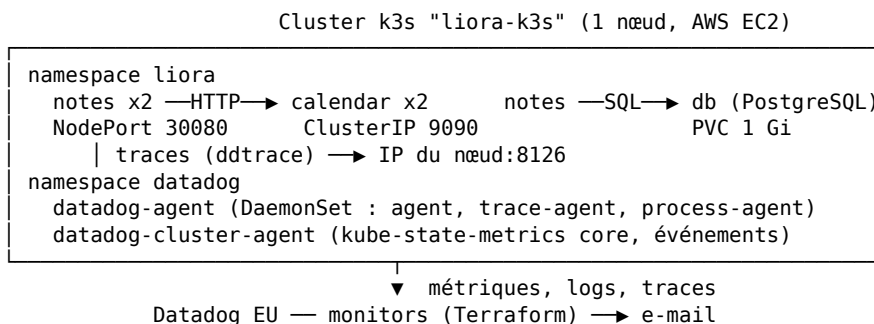
Dépôt GitHub	https://github.com/3imadsabri/datadog-k8s-monitoring
Application	https://github.com/datascientest/apm-datadog (notes, calendar, PostgreSQL)
Cluster	k3s 1 nœud « liora-k3s » sur AWS EC2 (c7i-flex.large, Ubuntu 24.04, eu-north-1)
Datadog	site EU (datadoghq.eu), agent installé par Helm, monitors créés par Terraform

1. Objectifs et résultat

L'énoncé demande de déployer l'application *apm-datadog* dans un cluster Kubernetes, puis de la superviser avec Datadog. Le tableau ci-dessous reprend chaque exigence, la façon dont elle a été réalisée et la preuve correspondante dans ce document.

Exigence	Réalisation	Preuve
Déployer l'application dans Kubernetes (Kompose suggéré)	Conversion Kompose, puis manifests corrigés (k8s/)	§3, capture 1
2 réplicas de notes et 2 de calendar	replicas: 2 dans les deux Deployments	captures 1 et 2
Traces APM des appels GET, POST, PUT, DELETE de notes	ddtrace-run + trace-agent Datadog (port 8126)	§5, captures 3 à 6
Alerte e-mail CPU > 60 %	Monitor Terraform cpu_high, testé avec stress-ng	§7, capture 11
Alerte e-mail RAM > 80 %	Monitor Terraform ram_high, testé avec stress-ng	§7, capture 12
Alerte e-mail pod non Running	Monitors pod_not_running et replicas_unavailable	§7, captures 8 à 10
Fichiers de configuration et logs des pods	Dépôt GitHub (k8s/, datadog/, terraform/) et logs/	§8

2. Architecture



Composant	Rôle
notes (Flask, 8080)	API REST GET/POST/PUT/DELETE /notes ; appelle calendar quand add_date=y
calendar (Flask, 9090)	renvoie une date aléatoire
db (PostgreSQL 16)	stocke les notes (base testdb), volume persistant 1 Gi
datadog-agent	un agent par nœud : métriques système, logs des conteneurs, réception des traces
datadog-cluster-agent	métriques d'état Kubernetes (phase des pods, réplicas) collectées une seule fois

Choix du cluster : k3s, la distribution Kubernetes légère vue en cours, sur une instance c7i-flex.large (2 vCPU, 4 Go) éligible à l'offre gratuite AWS. Une instance de 1 Go ne suffit pas pour k3s, l'agent Datadog et les 5 pods de l'application.

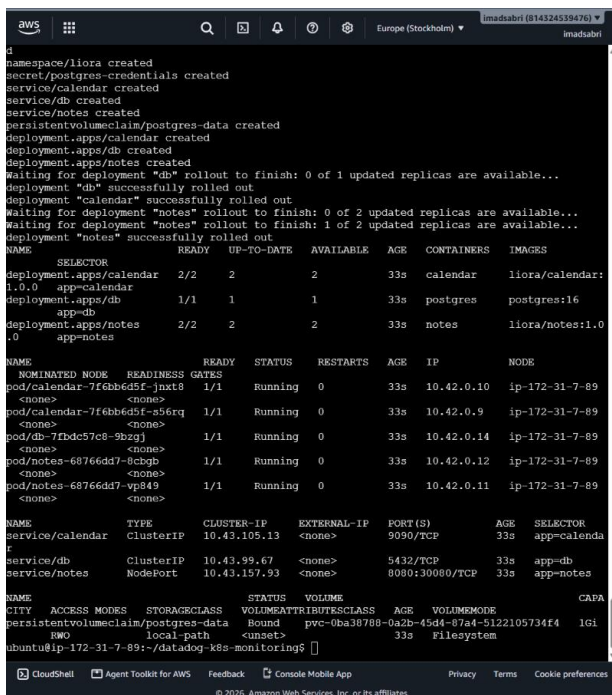
3. Du docker-compose aux manifests Kubernetes

Comme suggéré, le *docker-compose.yml* du dépôt a d'abord été converti avec **Kompose 1.34** (`kompose convert`). La sortie brute est conservée dans `kompose/`. Elle n'est pas déployable telle quelle ; chaque problème a été corrigé dans `k8s/` :

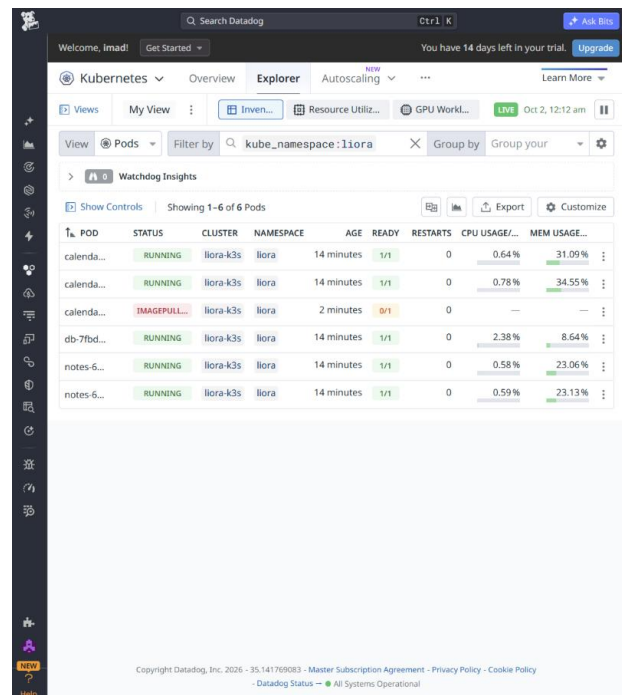
Problème dans la sortie Kompose	Correction
Services nommés <code>notes_app</code> et <code>calendar_app</code> : refusés par l'API (« _ » interdit)	Services <code>notes</code> et <code>calendar</code>
Aucun Service pour la base (pas de port publié dans le compose)	Service <code>db</code> sur le port 5432
L'agent Datadog converti en simple Deployment	Chart Helm officiel (DaemonSet + Cluster Agent)
<code>DD_AGENT_HOST=datadog</code> : nom inexistant dans le cluster	<code>DD_AGENT_HOST</code> = IP du nœud (Downward API <code>status.hostIP</code>)
1 réplica, ni ressources, ni sondes, mot de passe en clair	2 réplicas, requests/limits, readiness/liveness, Secret
<code>notes</code> plante si la base n'est pas encore prête	<code>initContainer wait-for-db</code>
<code>calendar</code> n'était pas instrumenté (pas de <code>ddtrace-run</code>)	<code>ddtrace-run</code> dans les deux Dockerfiles, <code>ddtrace==4.15.4</code>

Les images *liora/notes:1.0.0* et *liora/calendar:1.0.0* sont construites sur le nœud avec Docker puis importées dans le containerd de k3s (pas de registre nécessaire). Le déploiement complet tient en quelques scripts :

```
./scripts/00-install-prereqs.sh # k3s, Helm, Docker, Terraform, stress-ng
./scripts/01-build-images.sh   # images notes et calendar importées dans k3s
./scripts/02-deploy-app.sh     # kubectl apply -k k8s/ : db + calendar x2 + notes x2
./scripts/03-install-datadog.sh # agent Datadog (Helm), clé d'API dans un Secret
./scripts/04-create-monitors.sh # 4 monitors (Terraform)
```



Capture 1 — Pods du namespace *liora* : 2 réplicas *notes*, 2 réplicas *calendar* et PostgreSQL, tous Running.



Capture 2 — Les pods du cluster *liora-k3s* vus dans Datadog (Infrastructure > Kubernetes).

4. Installation de l'agent Datadog

L'agent est installé avec le chart Helm officiel (datadog/datadog) et le fichier `datadog/datadog-values.yaml`. Points importants :

- **site: datadoghq.eu** et **clusterName: liora-k3s** : toutes les métriques portent le tag `kube_cluster_name:liora-k3s`, utilisé par les monitors ;
- **apm.portEnabled: true** : le trace-agent écoute sur le port 8126 de chaque nœud ;
- **criSocketPath** vers le containerd de k3s et **kubelet.tlsVerify: false** (certificat auto-signé de k3s) ;
- **logs.containerCollectAll**, **processAgent** et **kubeStateMetricsCore** (phase des pods, réplicas) ;
- la clé d'API est saisie avec `read -s` et stockée dans un Secret Kubernetes : aucune clé dans le dépôt.

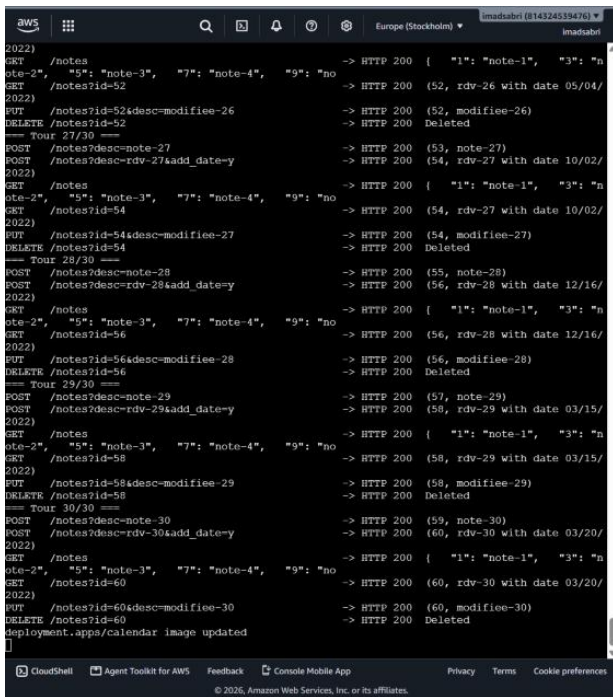
Les pods portent les labels d'**Unified Service Tagging**

(tags.datadoghq.com/env|service|version), copiés dans `DD_ENV`, `DD_SERVICE`, `DD_VERSION` : traces, logs et métriques d'un même service sont reliés (env:exam, service:notes / calendar, version:1.0.0).

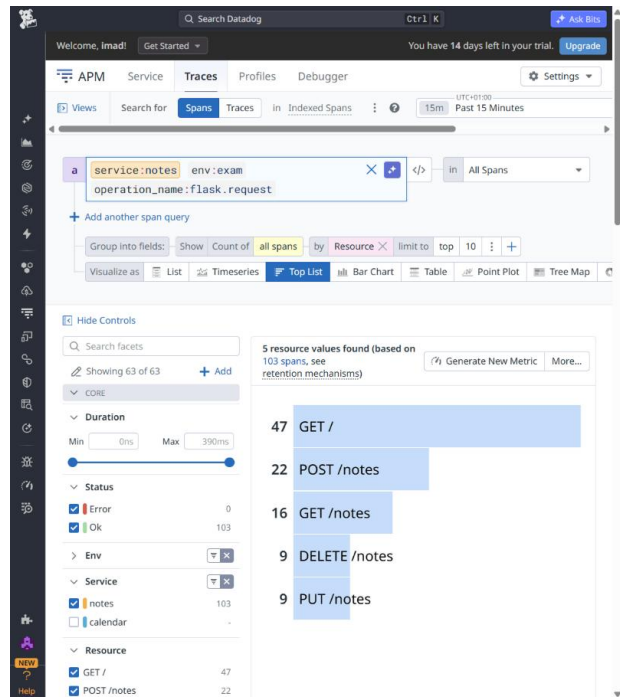
5. Traces APM des appels GET, POST, PUT et DELETE

Les deux services Flask sont lancés avec `ddtrace-run`, qui instrumente automatiquement Flask, `psycopg2` et `requests`. Chaque requête HTTP devient une trace ; chaque requête SQL et chaque appel à `calendar` devient un span. La bibliothèque envoie les traces à `DD_AGENT_HOST:8126`, c'est-à-dire au trace-agent du nœud.

Le script `scripts/generate-traffic.sh` envoie, à chaque tour, les quatre méthodes sur le NodePort 30080 : POST (création), POST avec `add_date=y` (appel à `calendar`), GET (liste et note), PUT (modification) et DELETE (suppression).

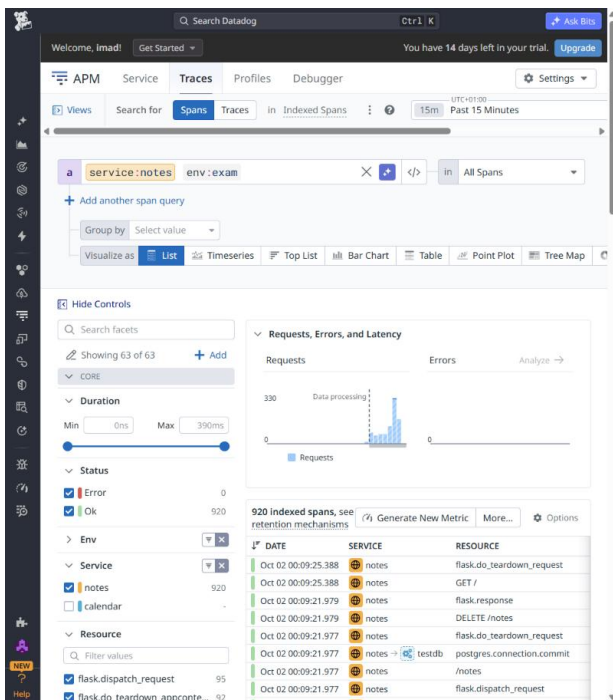


Capture 3 — Trafic généré : toutes les requêtes GET, POST, PUT et DELETE répondent HTTP 200.

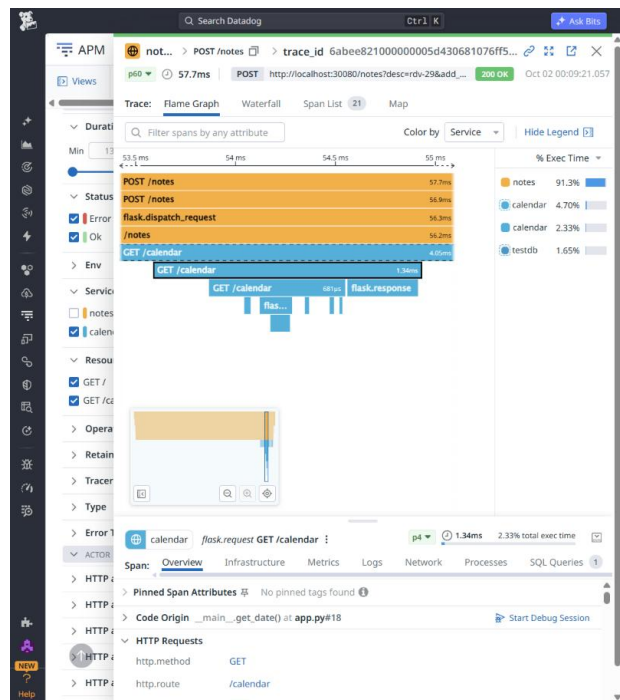


Capture 4 — APM : le service notes et ses ressources GET, POST, PUT et DELETE /notes.

Bilan observé dans Datadog : **920 spans, 0 erreur** ; par ressource : POST /notes 22, GET /notes 16, PUT /notes 9, DELETE /notes 9 (plus GET / utilisé par les sondes).



Capture 5 — Détail d'une trace : span Flask de notes et span de la requête PostgreSQL.



Capture 6 — Trace distribuée POST /notes?add_date=y (57,7 ms) : notes → GET /calendar → base testdb.

6. Monitors et alertes e-mail (Terraform)

Les monitors sont décrits en code dans `terraform/monitors.tf` (provider Datadog, *Monitoring as Code*) : ils sont versionnés, reproductibles sur un autre cluster en changeant la variable `cluster_name`, et chaque message notifie l'adresse e-mail passée en variable.

Monitor	Requête (fenêtre 5 min)	Seuils
CPU > 60 %	100 - avg:system.cpu.idle{kube_cluster_name:liora-k3s} by {host}	alerte 60, warning 50, rétablissement 55
RAM > 80 %	(1 - avg:system.mem.pct_usable{kube_cluster_name:liora-k3s} by {host}) * 100	alerte 80, warning 70, rétablissement 75
Pod non Running	sum:kubernetes_state.pod.status_phase{kube_namespace:liora, !pod_phase:running, !pod_phase:succeeded} by {pod_name,pod_phase}	> 0
Réplicas indisponibles	sum:kubernetes_state.deployment.replicas_unavailable{kube_namespace:liora} by {kube_deployment}	> 0

- **CPU** : 100 – idle inclut user, system et iowait (system.cpu.user seul ignorerait le noyau).
- **RAM** : `pct_usable` est la part réellement disponible ; `system.mem.used` compterait le cache disque et provoquerait de fausses alertes.
- **Pods** : un pod en `CrashLoopBackOff` reste en phase `Running` ; le second monitor (réplicas indisponibles) couvre ce cas.
- Le seuil de *warning* prévient avant l'alerte, et le seuil de rétablissement plus bas évite les alertes en rafale autour de la limite.

```
aws
+ restricted_roles = (known after apply)
+ tags = {
+   "env:exam",
+   "kube_cluster_name:liora-k3s",
+   "managed-by:terraform",
+   "team:monitoring",
+ }
+ type = "query alert"
+ monitor thresholds {
+   critical = "0"
+ }

Plan: 4 to add, 0 to change, 0 to destroy.

Changes to Outputs:
+ monitor_ids = {
+   cpu_high           = (known after apply)
+   pod_not_running    = (known after apply)
+   ram_high           = (known after apply)
+   replicas_unavailable = (known after apply)
+ }
datadog_monitor.replicas_unavailable: Creating...
datadog_monitor.pod_not_running: Creating...
datadog_monitor.cpu_high: Creating...
datadog_monitor.ram_high: Creating...
datadog_monitor.pod_not_running: Creation complete after 1s [id=125202600]
datadog_monitor.ram_high: Creation complete after 1s [id=125202602]
datadog_monitor.replicas_unavailable: Creation complete after 1s [id=125202599]
datadog_monitor.cpu_high: Creation complete after 1s [id=125202601]

Apply complete! Resources: 4 added, 0 changed, 0 destroyed.

Outputs:
monitor_ids = {
  "cpu_high" = "125202601"
  "pod_not_running" = "125202600"
  "ram_high" = "125202602"
  "replicas_unavailable" = "125202599"
}
monitor_ids = {
  "cpu_high" = "125202601"
  "pod_not_running" = "125202600"
  "ram_high" = "125202602"
  "replicas_unavailable" = "125202599"
}
ubuntu@ip-172-31-7-89:~/datadog-k8s-monitoring$
```

Capture 7 — terraform apply : création des 4 monitors.

7. Tests des alertes

Chaque alerte a été déclenchée volontairement avec scripts/test-alerts.sh. Heures indiquées en heure locale (UTC+1), le 2 octobre 2026.

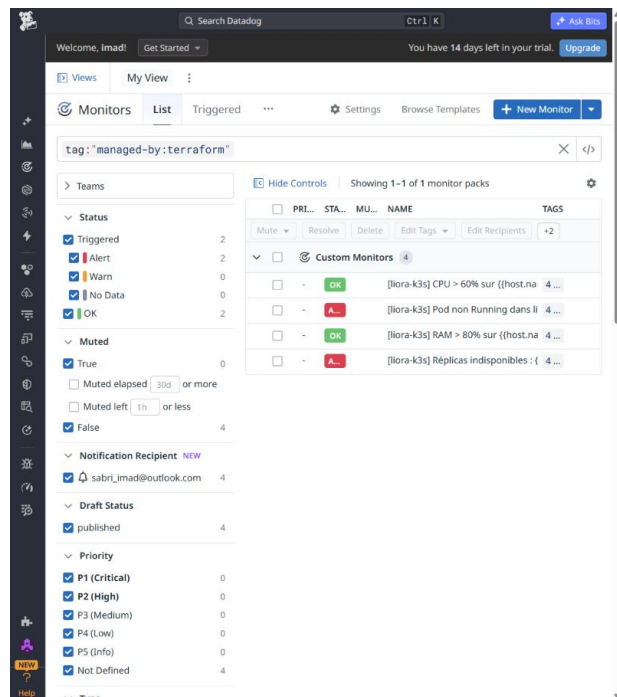
Test	Action	Résultat
Pod non Running	image inexistante sur calendar (test-alerts.sh pod) : le nouveau pod reste en ImagePullBackOff, phase Pending	Alerte pour le pod calendar-677fc54b6d-p79kp et pour le Deployment calendar, e-mail envoyé à 00:10 ; après pod-restore, le monitor des réplicas repasse à OK (voir l'ajustement ci-dessous pour le monitor des pods)
CPU > 60 %	stress-ng --cpu 0 --cpu-load 90 pendant 8 min	Warning à 00:19, alerte à 00:20 (CPU à 94 %), e-mail envoyé ; rétablissement à la fin du test
RAM > 80 %	stress-ng --vm réservant la mémoire disponible moins 12 % du total (1 282 Mo sur 3 814 Mo), pendant 8 min	RAM de 54 % à 90 % ; warning à 00:49, alerte à 00:50, e-mail envoyé ; retour à OK à 00:57

```

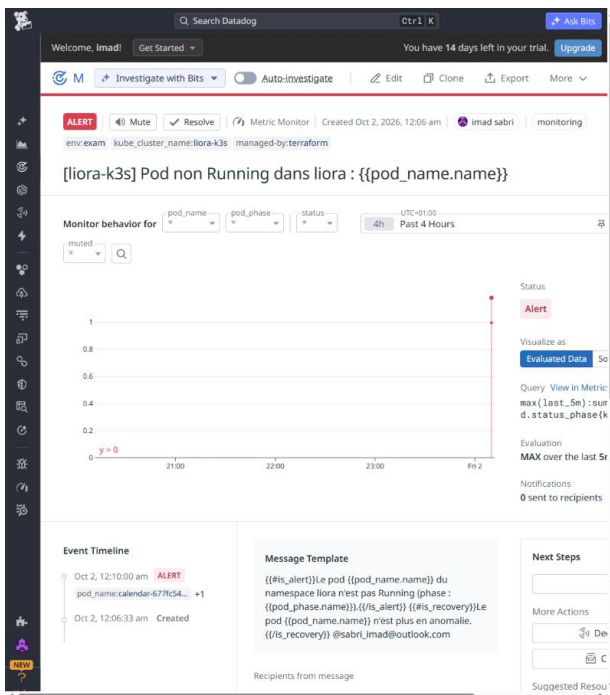
PUT /notes?id=54&desc=modifiee-27 -> HTTP 200 (54, modifiee-27)
DELETE /notes?id=54 -> HTTP 200 Deleted
Tour 28/30 ==
POST /notes?desc=note-28 -> HTTP 200 (55, note-28)
POST /notes?desc=rdv-28&add_date=y -> HTTP 200 (56, rdv-28 with date 12/16/2022)
GET /notes -> HTTP 200 ( "1": "note-1", "3": "note-2", "5": "note-3", "7": "note-4", "9": "note-5" )
PUT /notes?id=56&desc=modifiee-28 -> HTTP 200 (56, modifiee-28)
DELETE /notes?id=56 -> HTTP 200 Deleted
Tour 29/30 ==
POST /notes?desc=note-29 -> HTTP 200 (57, note-29)
POST /notes?desc=rdv-29&add_date=y -> HTTP 200 (58, rdv-29 with date 03/15/2022)
GET /notes -> HTTP 200 ( "1": "note-1", "3": "note-2", "5": "note-3", "7": "note-4", "9": "note-5" )
PUT /notes?id=58&desc=modifiee-29 -> HTTP 200 (58, modifiee-29)
DELETE /notes?id=58 -> HTTP 200 Deleted
Tour 30/30 ==
POST /notes?desc=note-30 -> HTTP 200 (59, note-30)
POST /notes?desc=rdv-30&add_date=y -> HTTP 200 (60, rdv-30 with date 03/20/2022)
GET /notes?id=60 -> HTTP 200 (60, rdv-30 with date 03/20/2022)
PUT /notes?id=60&desc=modifiee-30 -> HTTP 200 (60, modifiee-30)
DELETE /notes?id=60 -> HTTP 200 Deleted
Deployment.apps/calendar image updated
NAME Nominated Node Readiness Gates Status Restarts Age IP Node
calendar-677fc54b6d-p79kp 0/1 ImagePullBackOff 0 21s 10.42.0.18 ip-172-31-7-89 <none>
calendar-7f6bb6d5f-jnxt8 1/1 Running 0 11m 10.42.0.10 ip-172-31-7-89 <none>
calendar-7f6bb6d5f-s56rq 1/1 Running 0 11m 10.42.0.9 ip-172-31-7-89 <none>
dp-7f6bb6d5f-sb7g3 1/1 Running 0 11m 10.42.0.14 ip-172-31-7-89 <none>
notes-68766dd7-8chgb 1/1 Running 0 11m 10.42.0.12 ip-172-31-7-89 <none>
notes-68766dd7-vp849 1/1 Running 0 11m 10.42.0.11 ip-172-31-7-89 <none>
Alerte attendue d'ici 1 à 3 minutes. Restaurer avec : ./scripts/test-alerts.sh pod-restore

```

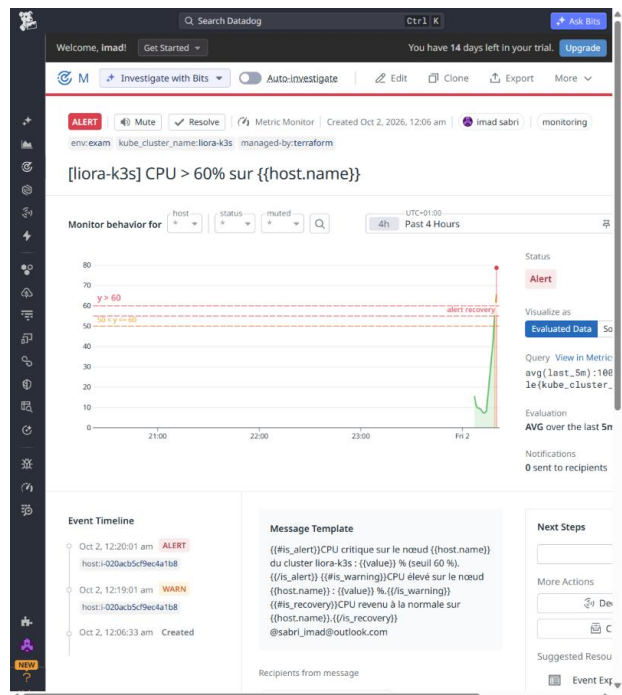
Capture 8 — Test : le nouveau pod calendar en ImagePullBackOff ; les 2 anciens réplicas continuent de servir.



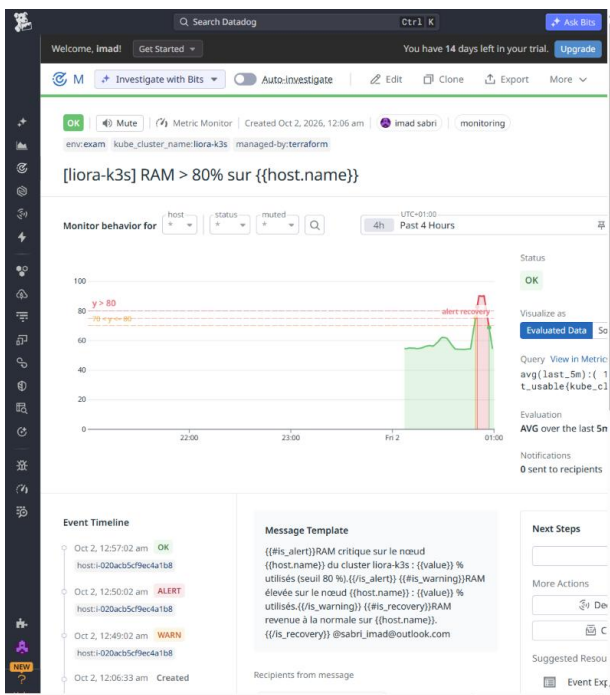
Capture 9 — Liste des monitors : les monitors de pods passent en ALERT.



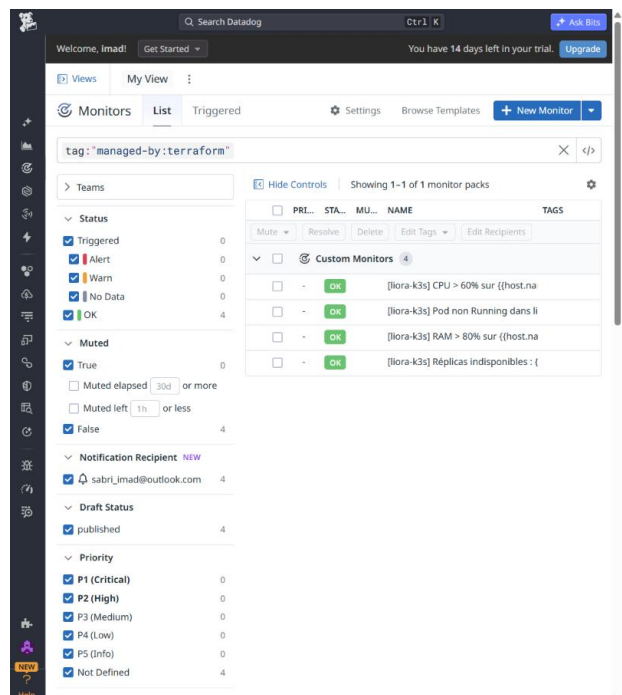
Capture 10 — Monitor « pod non Running » : alerte sur le pod calendar en phase Pending.



Capture 11 — Monitor CPU : passage en WARN puis ALERT pendant le test stress-ng.



Capture 12 — Monitor RAM : WARN à 00:49, ALERT à 00:50 (90 %), retour à OK à 00:57.



Capture 13 — Les 4 monitors créés par Terraform, tous revenus à OK après les tests.

Ajustements apportés pendant les tests

Monitor « pod non Running » : après le retour à la normale, le pod fautif est supprimé ; sa série de métriques disparaît au lieu de repasser à 0, et Datadog conserve le groupe dans son dernier état (ALERT). L'alerte de test a été résolue manuellement, et l'option `timeout_h = 1` a été ajoutée au monitor dans Terraform : un groupe qui n'envoie plus de données se résout alors automatiquement après une heure. Le monitor « réplicas indisponibles », lui, repasse bien à OK puisque le Deployment existe toujours.

Test RAM : le premier test RAM, avec un pourcentage passé directement à stress-ng, n'a fait monter la mémoire qu'à 62 %. Le script calcule désormais la quantité à réserver à partir de `/proc/meminfo` (mémoire disponible moins 12 % du total), ce qui amène l'utilisation vers 88 % sans risquer l'arrêt de pods par manque de mémoire.

8. Fichiers de configuration et logs des pods

Tous les fichiers sont sur le dépôt GitHub. Le script `scripts/collect-logs.sh` écrit les logs de chaque conteneur (initContainer compris) avec horodatage, l'état du cluster, le statut complet de l'agent Datadog et les logs du trace-agent.

Dossier / fichier	Contenu
k8s/	namespace, PostgreSQL (Secret, PVC, Deployment, Service), calendar, notes, kustomization
datadog/datadog-values.yaml	valeurs Helm de l'agent
terraform/	provider, variables, 4 monitors
app/	code de l'application et Dockerfiles
kompose/	docker-compose d'origine et sortie brute de Kompose
logs/etat-cluster.txt	nœud, Deployments, pods, Services, PVC, pods Datadog
logs/liora_<pod>_<conteneur>.log	logs des 5 pods (notes x2, calendar x2, db) et des initContainers
logs/datadog-agent-status.txt	statut de l'agent : checks, APM, logs
logs/datadog-trace-agent.log	logs du trace-agent (réception des traces)

Remarque : les premières lignes des logs de notes contiennent des erreurs *failed to send traces ... 8126*. Elles datent du démarrage de l'application, avant l'installation de l'agent Datadog : `ddtrace` ne trouvait pas encore de trace-agent. Elles cessent dès que l'agent est en place (statut APM Agent : Running, receiver 0.0.0.0:8126).

Extrait des logs

```
$ cat logs/etat-cluster.txt (extrait, 2026-10-01 23:53:53 UTC)
```

NAME	READY	UP-T0-DATE	AVAILABLE	IMAGES
deployment.apps/calendar	2/2	2	2	liora/calendar:1.0.0
deployment.apps/db	1/1	1	1	postgres:16
deployment.apps/notes	2/2	2	2	liora/notes:1.0.0

NAME	READY	STATUS	RESTARTS	AGE
pod/calendar-7f6bb6d5f-jnxt8	1/1	Running	0	55m
pod/calendar-7f6bb6d5f-s56rq	1/1	Running	0	55m
pod/db-7fbdc57c8-9bzgj	1/1	Running	0	55m
pod/notes-68766dd7-8cbgb	1/1	Running	0	55m
pod/notes-68766dd7-vp849	1/1	Running	0	55m
datadog-agent-82m9d	3/3	Running	0	48m
datadog-agent-cluster-agent-cb6c59689-rqpdm	1/1	Running	0	48m
datadog-agent-operator-7bf7794596-mznn4	1/1	Running	0	48m

```
$ grep -E 'PUT|DELETE' logs/liora_notes-68766dd7-8cbgb_notes.log | tail -4
```

```
INFO:werkzeug:10.42.0.1 - - [01/Oct/2026 23:31:57] "PUT /notes?id=74&desc=modifiee-7 HTTP/1.1" 200 -  
INFO:werkzeug:10.42.0.1 - - [01/Oct/2026 23:31:57] "DELETE /notes?id=74 HTTP/1.1" 200 -  
INFO:werkzeug:10.42.0.1 - - [01/Oct/2026 23:32:01] "PUT /notes?id=78&desc=modifiee-9 HTTP/1.1" 200 -  
INFO:werkzeug:10.42.0.1 - - [01/Oct/2026 23:32:01] "DELETE /notes?id=78 HTTP/1.1" 200 -
```

9. Conclusion

L'application est déployée dans un cluster k3s avec 2 réplicas de notes et 2 de calendar. Datadog collecte les métriques du nœud et du cluster, les logs des conteneurs et les traces APM des quatre méthodes HTTP, y compris la trace distribuée notes → calendar → PostgreSQL. Les alertes CPU, RAM et pod non Running ont été déclenchées et reçues par e-mail.

Enseignements : Kompose accélère la conversion mais sa sortie doit être relue et corrigée ; décrire les monitors en Terraform les rend reproductibles ; le choix des métriques (CPU non idle, mémoire utilisable, phase des pods et réplicas indisponibles) évite à la fois les alertes manquées et les fausses alertes.

Pour superviser plusieurs clusters (multi-cloud), il suffit d'installer le même chart avec un `clusterName` différent et d'appliquer le même code Terraform par cluster.